

Java Viva Questions and Answers

Multithreading (Q1–15) • File Handling (Q16–30)

A. Multithreading in Java

Q1. What is a thread in Java?

A **thread** is the smallest unit of execution within a program. It is a lightweight sub-process that runs independently but shares the memory (heap) of its parent process. Every Java program has at least one thread, the **main** thread, which runs the `main()` method. In Java, a thread is represented by the `java.lang.Thread` class.

Q2. What is multithreading?

Multithreading is the ability of a program to execute two or more threads concurrently, so that different parts of the program run at the same time (on multi-core CPUs) or are interleaved by the scheduler (on a single core). Example: a music player downloads a song in one thread while playing another song in a second thread.

Q3. Why is multithreading used in Java applications?

- **Better CPU utilisation** - the CPU does not sit idle while one task waits for I/O.
- **Responsiveness** - a GUI stays responsive while a long task runs in the background.
- **Faster execution** - big tasks can be divided and run in parallel on multiple cores.
- **Resource sharing** - threads share memory, so they use fewer resources than processes.
- **Simplified modelling** - servers handle many client requests at once (one thread per request).

Q4. What is the difference between a process and a thread?

Process	Thread
An independent program in execution.	A lightweight unit of execution inside a process.
Has its own separate memory space.	Shares the memory (heap) of the process; has its own stack.
Heavyweight; creation and context switching are costly.	Lightweight; creation and context switching are cheap.
Inter-process communication is complex (pipes, sockets).	Communication is easy through shared objects.
Crash of one process usually does not affect others.	An uncaught problem in one thread can affect the whole process.

Q5. How can you create a thread in Java?

There are two classic ways (plus modern ones):

- **By extending the Thread class** and overriding `run()`.
- **By implementing the Runnable interface** and passing the object to a Thread constructor.
- Also: Callable with `ExecutorService`, and lambda expressions with `Runnable`.

```
// 1) Extending Thread
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running: " + getName());
    }
}
```

```

}

// 2) Implementing Runnable
class MyTask implements Runnable {
    public void run() {
        System.out.println("Runnable running");
    }
}

public class Demo {
    public static void main(String[] args) {
        new MyThread().start();
        new Thread(new MyTask()).start();
        new Thread(() -> System.out.println("Lambda thread")).start();
    }
}

```

Q6. What is the difference between extending Thread and implementing Runnable?

Extending Thread	Implementing Runnable
Class cannot extend any other class (Java has no multiple inheritance of classes).	Class can still extend another class and implement Runnable.
Each thread gets its own object.	The same Runnable object can be shared by many threads.
Tightly couples the task with the thread mechanism.	Separates the task (what to run) from the runner (Thread). Better design.
Inherits all Thread methods.	Only has the single run() method.

Preferred: Runnable, because it is more flexible and follows good OOP design.

Q7. What is the purpose of the start() method?

start() creates a **new thread of execution**, moves the thread from the *New* state to *Runnable*, and then the JVM calls the thread's run() method on that new thread. A thread can be started only once; calling start() again throws `IllegalThreadStateException`.

Q8. What is the difference between start() and run()?

start()	run()
Creates a new thread and runs run() in it.	Just a normal method call; runs in the current (caller) thread.
Achieves real multithreading.	No new thread is created, so no multithreading.
Can be called only once per thread.	Can be called any number of times.
Defined in Thread class.	Defined in Runnable; overridden by you.

```

Thread t = new Thread(() -> System.out.println(Thread.currentThread().getName()));
t.start(); // prints Thread-0 (new thread)
t.run();  // prints main (same thread, normal call)

```

Q9. What is the life cycle of a thread?

A thread passes through these states (`Thread.State` enum):

- **NEW** - thread object created but start() not yet called.
- **RUNNABLE** - ready to run or currently running; waiting for CPU time from the scheduler.
- **BLOCKED** - waiting to acquire a monitor lock (e.g., entering a synchronized block).
- **WAITING** - waiting indefinitely for another thread (wait(), join()).

- **TIMED_WAITING** - waiting for a fixed time (`sleep(ms)`, `wait(ms)`, `join(ms)`).
- **TERMINATED** - `run()` has finished or the thread died due to an exception.

```
NEW --start()--> RUNNABLE <--> (BLOCKED / WAITING / TIMED_WAITING)
      |
      +--run() ends--> TERMINATED
```

Q10. What is the purpose of `sleep()`?

`Thread.sleep(milliseconds)` pauses the **currently executing thread** for the given time without releasing any locks it holds. It moves the thread to `TIMED_WAITING` and is used for delays, simulations and giving other threads a chance to run. It throws the checked `InterruptedException`.

```
try {
    Thread.sleep(1000); // sleep for 1 second
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

Q11. What is the difference between `sleep()` and `wait()`?

<code>sleep()</code>	<code>wait()</code>
Defined in Thread class (static method).	Defined in Object class (instance method).
Does NOT release the lock it holds.	RELEASES the lock (monitor) on the object.
Wakes up after the time expires (or on interrupt).	Wakes on <code>notify()/notifyAll()</code> , timeout or interrupt.
Can be called anywhere.	Must be called inside a synchronized block/method, else <code>IllegalMonitorStateException</code> .
Used for pausing execution.	Used for inter-thread communication.

Q12. What is thread synchronization?

Synchronization is the mechanism that controls access of multiple threads to a shared resource so that only **one thread at a time** can use it. It prevents data inconsistency and race conditions. It is implemented using the `synchronized` keyword (monitor locks), and also through Lock classes and atomic variables in `java.util.concurrent`.

Q13. Why is the `synchronized` keyword used?

- To make a method or block **thread-safe** - only one thread can hold the object's lock and execute it at a time.
- To prevent **race conditions** and keep shared data consistent.
- To guarantee **visibility** - changes made by one thread are visible to others after the lock is released.
- It can be applied to an instance method, a static method (class-level lock) or a block.

```
class Counter {
    private int count = 0;

    public synchronized void increment() { // synchronized method
        count++;
    }

    public void incrementBlock() {
        synchronized (this) { // synchronized block
            count++;
        }
    }

    public int getCount() { return count; }
}
```

Q14. What is a race condition?

A **race condition** occurs when two or more threads access and modify shared data at the same time and the final result depends on the unpredictable order in which the threads execute. Example: two threads running `count++` (which is read-modify-write, not atomic) may both read the same old value, so one increment is lost. **Solution:** synchronization, locks, or atomic classes like `AtomicInteger`.

Q15. What is the difference between `join()` and `sleep()`?

<code>join()</code>	<code>sleep()</code>
Makes the current thread wait until ANOTHER thread finishes (<code>t.join()</code>).	Makes the current thread pause for a FIXED time.
Instance method of Thread.	Static method of Thread.
Waiting ends when the target thread terminates (or timeout).	Waiting ends when the time elapses.
Used for ordering/coordination of threads.	Used for delays.

```
Thread t1 = new Thread(() -> System.out.println("Worker done"));
t1.start();
t1.join();                // main waits here until t1 finishes
System.out.println("Main continues after t1");
```

B. File Handling in Java

Q16. What is file handling in Java?

File handling means creating, reading, writing, updating, deleting and managing files stored on disk from a Java program. It lets a program store data permanently (persistent storage) instead of losing it when the program ends. Java performs file handling through **streams** - a stream is a sequence of data flowing from a source to a destination.

Q17. Which package is commonly used for file handling in Java?

java.io is the main package (File, FileReader, FileWriter, FileInputStream, FileOutputStream, BufferedReader, etc.). The newer java.nio.file package (Files, Path, Paths) offers a more modern API.

```
import java.io.*;           // classic file handling
import java.nio.file.*;     // NIO.2 (Files, Path)
```

Q18. What is the purpose of the File class?

java.io.File is an **abstract representation of a file or directory path**. It is used to get information about files/directories (name, path, size, permissions), check existence, create, delete, rename and list contents. **Important:** the File class does NOT read or write file content - for that you need streams, readers or writers.

```
File f = new File("data.txt");
System.out.println(f.length()); // size in bytes
System.out.println(f.canRead());
f.delete();
```

Q19. How do you create a new file using Java?

Use createNewFile() of the File class. It returns **true** if the file was created and **false** if it already exists. It throws IOException.

```
import java.io.*;

public class CreateFile {
    public static void main(String[] args) {
        try {
            File f = new File("myfile.txt");
            if (f.createNewFile())
                System.out.println("File created: " + f.getName());
            else
                System.out.println("File already exists.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Alternatives: new FileWriter("x.txt") or Files.createFile(Paths.get("x.txt")) also create files.

Q20. How do you check whether a file exists?

Use the exists() method of the File class. It returns true if the file or directory exists. You can also use isFile() / isDirectory(), or Files.exists(path) in NIO.

```
File f = new File("myfile.txt");
if (f.exists()) System.out.println("Exists");
else            System.out.println("Does not exist");
```

Q21. How do you obtain the name and path of a file?

Method	Returns
getName()	Name of the file (e.g., myfile.txt).
getPath()	The path string used when the File object was created (may be relative).
getAbsolutePath()	Full absolute path of the file.
getParent()	Parent directory path.
length()	File size in bytes.

```
File f = new File("myfile.txt");
System.out.println("Name: " + f.getName());
System.out.println("Path: " + f.getPath());
System.out.println("Absolute path: " + f.getAbsolutePath());
```

Q22. What is the difference between FileReader and FileWriter?

FileReader	FileWriter
Used to READ character data from a file.	Used to WRITE character data to a file.
Extends InputStreamReader (Reader hierarchy).	Extends OutputStreamWriter (Writer hierarchy).
Main method: read() - returns one character (int), -1 at end of file.	Main methods: write(), append(), flush().
Throws FileNotFoundException if the file does not exist.	Creates the file if it does not exist; overwrites by default (use new FileWriter(name, true) to append).

```
FileWriter fw = new FileWriter("a.txt");
fw.write("Hello Java");
fw.close();

FileReader fr = new FileReader("a.txt");
int ch;
while ((ch = fr.read()) != -1) System.out.print((char) ch);
fr.close();
```

Q23. What is the difference between character streams and byte streams?

Byte Streams	Character Streams
Handle data in 8-bit bytes.	Handle data in 16-bit Unicode characters.
Base classes: InputStream and OutputStream.	Base classes: Reader and Writer.
Best for binary data: images, audio, video, PDFs.	Best for text data: .txt, .csv, source code.
Examples: FileInputStream, FileOutputStream.	Examples: FileReader, FileWriter, BufferedReader.
No character encoding conversion.	Automatically handle character encoding.

Q24. What is the purpose of FileInputStream?

FileInputStream reads **raw bytes** from a file. It is used to read binary files such as images, audio or any file where byte-level access is needed. read() returns the next byte as an int (0-255) or -1 at end of file.

```
FileInputStream fis = new FileInputStream("photo.jpg");
int b;
while ((b = fis.read()) != -1) {
    // process byte b
}
fis.close();
```

Q25. What is the purpose of `FileOutputStream`?

`FileOutputStream` writes **raw bytes** to a file. It is used to write binary data such as images or to copy files. It creates the file if it does not exist and overwrites by default; new `FileOutputStream(name, true)` appends instead.

```
FileOutputStream fos = new FileOutputStream("out.dat");
fos.write(65); // writes byte 65 ('A')
fos.write("Hello".getBytes()); // writes a byte array
fos.close();
```

Q26. What is the difference between `BufferedReader` and `FileReader`?

FileReader	BufferedReader
Reads directly from the file, one character (or small array) at a time.	Wraps another Reader and uses an internal buffer (default 8192 chars).
Slower for large files because of frequent disk access.	Faster - fewer disk reads.
Has <code>read()</code> only (no <code>readLine()</code>).	Provides <code>readLine()</code> to read a whole line at once.
Can be used alone.	Cannot read a file alone; it needs an underlying Reader like <code>FileReader</code> .

```
BufferedReader br = new BufferedReader(new FileReader("a.txt"));
```

Q27. How do you read a file line by line in Java?

Wrap a `FileReader` in a `BufferedReader` and call `readLine()` until it returns **null**. (Also possible with `Scanner` and `hasNextLine()`, or `Files.readAllLines()`.)

```
import java.io.*;

public class ReadLines {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader("a.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Using `Scanner`:

```
Scanner sc = new Scanner(new File("a.txt"));
while (sc.hasNextLine()) System.out.println(sc.nextLine());
sc.close();
```

Q28. How do you write data into a text file?

Use `FileWriter` (optionally wrapped in `BufferedWriter` or `PrintWriter`). Pass `true` as the second argument to append instead of overwrite.

```
import java.io.*;

public class WriteFile {
    public static void main(String[] args) {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt"))) {
            bw.write("Hello, Java!");
            bw.newLine();
            bw.write("File handling is easy.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

```

    }
}

// Append mode:  new FileWriter("out.txt", true)

```

Q29. Why should files be closed after reading or writing?

- **Releases system resources** - file handles/descriptors are limited; unclosed streams cause resource leaks.
- **Flushes buffered data** - data written may stay in a buffer and never reach the file if not closed/flushed.
- **Avoids file locking** - the file may stay locked and cannot be deleted/modified by other programs.
- **Prevents data corruption/loss** and memory leaks.
- Best practice: use **try-with-resources** (Java 7+), which closes the resource automatically.

```

try (FileReader fr = new FileReader("a.txt")) {
    // use fr; it is closed automatically
} catch (IOException e) {
    e.printStackTrace();
}

```

Q30. What exceptions can occur during file handling, and how can they be handled?

Exception	When it occurs
FileNotFoundException	File does not exist or cannot be opened (e.g., wrong path, no permission).
IOException	General I/O failure while reading, writing or closing (parent of FileNotFoundException).
SecurityException	Security manager denies access to the file.
EOFException	Unexpected end of file while reading (e.g., DataInputStream).
UncheckedIOException	Wraps IOException in NIO/stream APIs such as Files.lines().

Handling: use try-catch, finally or try-with-resources, or declare with throws. Catch the more specific exception (FileNotFoundException) before the general one (IOException).

```

try (BufferedReader br = new BufferedReader(new FileReader("missing.txt"))) {
    System.out.println(br.readLine());
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    System.out.println("I/O error: " + e.getMessage());
} finally {
    System.out.println("Done");
}

```